

Technical Feature Specification: ADAL

1. Document Ingestion & OCR Processing

This module handles the uploading, cleaning, and digitization of legal documents (PDF/Images) into machine-readable text, supporting both English and Urdu scripts.

- **Key Techniques:**
 - **Preprocessing Pipeline:** Implementation of grayscale conversion, median blur for noise removal, and adaptive thresholding (Otsu's method) to enhance character visibility.
 - **Urdu Optimization:** Usage of CLAHE (Contrast Limited Adaptive Histogram Equalization) and morphological operations (kernel closing) to connect broken Urdu characters before recognition.
 - **Language Detection:** Character frequency analysis to automatically distinguish between English and Urdu segments, or mixed-mode processing.
- **Tech Stack:**
 - **Engine:** Tesseract OCR (v5) with `eng` and `urd` trained data.
 - **Preprocessing:** OpenCV (`cv2`) and Pillow (`PIL`).
 - **PDF Handling:** `pdf2image` (with Poppler utils) for converting PDF pages to processable image streams.

2. Citation Verification & Validation

This feature extracts legal citations (case laws, statutes) from the text, normalizes them, and validates their authenticity against a structured legal database.

- **Key Techniques:**
 - **Pattern Matching:** Utilization of Regex and NLP (spaCy) to identify citation patterns (e.g., "PLD 2020 SC 1").
 - **Validation Logic:** Cross-referencing extracted citations against a PostgreSQL database. The system assigns a confidence score based on the existence of the record and metadata matching.
 - **Normalization:** Standardizing various citation formats into a unified structure for consistent database querying.
- **Tech Stack:**
 - **NLP:** spaCy, NLTK, or Regex.
 - **Database:** PostgreSQL (Relational store for valid judgments).
 - **Search:** Elasticsearch for rapid lookup and filtering.

3. AI-Driven Legal Research (RAG & Summarization)

This module uses Retrieval-Augmented Generation (RAG) to provide context-aware summaries and retrieve relevant case precedents without hallucination.

- **Key Techniques:**

- **RAG Pipeline:** retrieving relevant legal context from a vector store before passing the query to the LLM for answer generation.
- **Semantic Search:** Generating vector embeddings of legal texts to perform cosine similarity searches, allowing the retrieval of conceptually similar cases rather than just keyword matches.
- **Abstractive Summarization:** Using Transformer models to generate concise summaries of lengthy judgments.
- **Tech Stack:**
 - **Models:** Hugging Face Transformers (Legal-BERT, Sentence Transformers) for embeddings and understanding.
 - **Vector Database:** FAISS, Pinecone, or Qdrant for storing embeddings.
 - **Orchestration:** LangChain or LlamaIndex.

4. Smart Legal Drafting Assistant

An interactive tool that assists lawyers in generating legal drafts (contracts, notices) using AI, based on specific prompts and case context.

- **Key Techniques:**
 - **Context Injection:** Building a context window composed of the user's prompt + retrieved relevant case laws (from the RAG module) to guide the LLM.
 - **Template Filling:** Using prompt engineering to ensure the output adheres to formal legal structures (Facts, Issues, Arguments).
 - **Draft Export:** Converting generated HTML/Markdown content into standard formats like PDF or Word.
- **Tech Stack:**
 - **LLM:** Fine-tuned Large Language Models (via Hugging Face).
 - **Editor:** Quill.js (Rich Text Editor) for frontend editing.
 - **Backend:** FastAPI (Python) for handling generation requests.

5. System Infrastructure & DevOps

The underlying architecture ensuring scalability, security, and performance.

- **Tech Stack:**
 - **Backend:** FastAPI (Python 3.9+).
 - **Frontend:** React.js (v18) with TypeScript and Material-UI.
 - **Asynchronous Tasks:** Celery + Redis (for handling heavy OCR/ML tasks in the background).
 - **Containerization:** Docker + Docker Compose.
 - **Storage:** S3 for document retention.

Tech-Stack: ADAL

1. Cloud Storage (Alternative to AWS S3)

For storing the uploaded legal documents (PDFs/Images) and OCR results.

- **Best Option: Supabase Storage** (Free Tier: 1GB storage, 2GB transfer).
 - *Why:* It provides an S3-compatible API, is easy to integrate with a Postgres database, and handles file permissions well.
- **Alternative: Cloudinary** (Best for images/OCR).
 - *Why:* Excellent for image transformations (e.g., preprocessing for OCR before your backend even touches it).
- **Alternative: Firebase Storage** (Google).
 - *Why:* Generous free tier (5GB), but requires a slightly different SDK setup than S3.

2. Database (Alternative to AWS RDS/EC2)

For your structured legal data (Users, Citations, Metadata).

- **Best Option: Supabase Database or Neon.**
 - *Why:* Both offer fully managed **PostgreSQL** on a free tier. Supabase also gives you a built-in GUI to view your data, which is great for FYP demos.
- **Alternative: Render PostgreSQL.**
 - *Why:* Good integration if you host your backend on Render, but the free tier database expires after 90 days (usually enough for an FYP).

3. Backend Hosting (Alternative to AWS EC2/Lambda)

For hosting your FastAPI (Python) application and Tesseract OCR engine.

- **Best Option: Render** (Free Tier).
 - *Why:* Supports Python/FastAPI and allows you to deploy Docker containers (essential for installing Tesseract/Poppler dependencies).
 - *Note:* The free tier spins down after inactivity (1 min warm-up time), which is acceptable for an FYP.
- **Alternative: Hugging Face Spaces** (Free Tier).
 - *Why:* Fantastic for AI/ML demos. You can host your FastAPI app inside a Space (using Docker) with decent CPU resources for free. It is specifically optimized for ML apps.
- **Alternative: Railway** (Trial/Cheap).
 - *Why:* Not indefinitely free, but offers a \$5 credit which lasts a long time for student projects. Very easy deployment.

4. Vector Database (For RAG/Embeddings)

For storing the embeddings of legal precedents.

- **Best Option: Pinecone** (Free "Starter" Tier).
 - *Why:* Cloud-hosted, very fast, and the free tier supports up to 100k vectors (plenty for FYP).
- **Alternative: ChromaDB** (Local/Self-hosted).
 - *Why:* It runs locally inside your Python application container. **Completely free** and zero network latency, making it perfect for the "Context Injection" feature.

5. Frontend Hosting

For your React.js application.

- **Best Option: Vercel or Netlify.**
 - *Why:* Both are industry standards, completely free for hobby projects, and offer automatic deployments from GitHub.

6. OCR & ML Model Processing

Since running heavy OCR (Tesseract) and LLMs (Hugging Face) on a free cloud CPU can be slow or crash:

- **Strategy A (Hybrid):** Host the Frontend (Vercel) and Database (Supabase) in the cloud, but **run the heavy Backend/AI locally** on your laptop (localhost) during the presentation. Use **Ngrok** to expose your local API to the cloud frontend if needed.
- **Strategy B (Hugging Face):** Use Hugging Face Inference API (Free Tier) for the LLM parts instead of hosting the model yourself.

Technical Timeline: ADAL

Phase 2: Intelligent Analysis & Search (Weeks 1-6)

Goal: Transition from simple text extraction to "understanding" the law using AI models.

- **Weeks 1-2: AI Model Integration**
 - **Task:** Integrate **model** for legal text classification and understanding.
 - **Task:** Implement **Sentence Transformers** to convert legal text into vector embeddings.
 - **Action:** Create a Python script that takes OCR text and outputs a vector array.
- **Weeks 3-4: Advanced Search (Semantic Layer)**
 - **Task:** Implement **Semantic Search** functionality.

- **Task:** Replace/Augment basic keyword search with vector-based retrieval using your chosen vector database (e.g., Pinecone or ChromaDB).
- **Action:** Ensure that searching "breach of contract" retrieves cases that mention "violation of agreement" even if exact keywords differ.
- **Weeks 5-6: Background Processing**
 - **Task:** Implement **Celery + Redis** for managing heavy tasks.
 - **Action:** Move OCR and AI processing to background queues so the frontend doesn't freeze while a file uploads.

Phase 3: Collaboration & RAG (Weeks 7-12)

Goal: Enable the "Smart Assistant" features and multi-user collaboration.

- **Weeks 7-8: Retrieval-Augmented Generation (RAG)**
 - **Task:** Connect your Vector Database (FAISS/Pinecone) to the LLM generation pipeline.
 - **Action:** Build the logic where the system fetches top-3 relevant case precedents and feeds them into the LLM to generate a draft.
- **Weeks 9-10: Real-Time Features**
 - **Task:** Implement **WebSockets (or Firebase)** for real-time document editing.
 - **Action:** Allow multiple users (e.g., a lawyer and a researcher) to view/edit the extracted text simultaneously.
- **Weeks 11-12: System Optimization**
 - **Task:** Optimize OCR preprocessing pipelines to handle lower-quality images.
 - **Task:** Refine the prompt engineering for the Legal Draft Assistant to ensure it follows strict legal formatting.

Phase 4: Testing & Deployment (Weeks 13-16)

Goal: Production-ready deployment using the free resources.

- **Weeks 13-14: Integration Testing**
 - **Task:** Conduct Unit and Integration testing for the new AI and Search modules.
 - **Task:** verify that **HTTPS encryption** is active for all communications.
- **Weeks 15-16: Final Deployment**
 - **Task:** Dockerize the final application.
 - **Task:** Deploy Frontend to **Vercel** and Backend to **Render** (or your chosen free alternative).
 - **Task:** Perform final "User Acceptance Testing" (UAT) to ensure the system analyzes a standard case document within the target 10-15 seconds.